

*Mastering Operators:*  
*Math, Logic, Comparison &*  
*Conditional*

A yellow square containing the letters 'JS' in a bold, black, sans-serif font, representing the JavaScript logo.

JS

# Introduction to Operators

## ❖ What Are Operators?

- Operators are symbols or keywords that perform operations on data (operands).  
They form the foundation of programming by enabling calculations, comparisons, and logical decisions.

### Categories of Operators:

- **Arithmetic Operators**
- **Comparison Operators**
- **Logical Operators**
- **Assignment Operators**

The image shows the JavaScript logo, which consists of the letters 'JS' in a bold, dark blue font, centered on a bright yellow square background.

# Arithmetic Operators

Arithmetic operators perform mathematical operations on numeric data.

Types of Arithmetic operators:

- Addition (+), Subtraction (-), Multiplication(\*), Division(/)
- Modulus (%), Exponential (\*\*)
- Increment Operator (++), Decrement Operator (--)

Example:

```
let a = 10, b = 3;  
console.log(a + b);    // Outputs: 13  
console.log(a % b);    // Outputs: 1  
console.log(2 ** 3);   // Outputs: 8
```

```
let x = 5;  
x++;  
console.log(x);        // Outputs: 6
```



# Comparison Operators

Comparison operators are used to compare two values and return a Boolean (**true** or **false**).

Types of Comparison operators:

- Equal to (==), Strictly equal (===), Not equal (!=), Strictly not equal(!==)
- Greater than (>), Less than (<)
- Greater than or equal to (>=), Less than or equal to (<=)

Example:

```
let age = 18;
```

```
console.log(age >= 18); // Outputs: true
```

```
console.log(5 === "5"); // Outputs: false
```

```
console.log(10 > 5); // Outputs: true
```

A yellow square containing the letters 'JS' in a bold, dark blue, sans-serif font.

# Logical Operators

Logical operators combine multiple conditions or invert them, returns a Boolean (**true** or **false**).

Types of Logical operators:

- Logical AND (&&), Logical OR (||), Logical NOT (!)

Example:

```
let x = 10, y = 20;  
console.log(x > 5 && y > 15); // Outputs: true  
console.log(x > 15 || y > 15); // Outputs: true  
console.log(!(x > 15)); // Outputs:
```

A yellow square containing the letters 'JS' in a bold, black, sans-serif font, representing JavaScript.

# Assignment Operators

Assignment operators assign values to variables, often in conjunction with arithmetic.

Types of Comparison operators:

- Assigns value (=) , Add & assign (+=), Subtract & assign (-=)
- Multiply & assign (\*=), Divide & assign (/=), Modulus and assign(%=)

Example:

```
let num = 10;  
num += 5; // Same as: num = num + 5  
console.log(num); // Outputs: 15  
num *= 2; // Same as: num = num * 2  
console.log(num); // Outputs: 30
```

A yellow square containing the letters 'JS' in a bold, dark blue, sans-serif font.

# Operator Precedence

When multiple operators are used, JavaScript determines the order of execution based on

| Precedence Level | Operators                                                                       | Description           |
|------------------|---------------------------------------------------------------------------------|-----------------------|
| 1                | <code>()</code>                                                                 | Parentheses           |
| 2                | <code>**</code>                                                                 | Exponentiation        |
| 3                | <code>*</code> , <code>/</code> , <code>%</code>                                | Multiplication, etc.  |
| 4                | <code>+</code> , <code>-</code>                                                 | Addition, Subtraction |
| 5                | <code>&gt;</code> , <code>&lt;</code> , <code>&gt;=</code> , <code>&lt;=</code> | Comparison            |
| 6                | <code>==</code> , <code>!=</code> , <code>===</code>                            | Equality              |
| 7                | <code>&amp;&amp;</code>                                                         | Logical AND           |
| 8                | <code>  </code>                                                                 | Logical OR            |
| 9                | <code>=</code> , <code>+=</code> , <code>-=</code>                              | Assignment            |



# Operator Precedence

The below are the few examples to understand the precedence

Example:

```
let result = 3 + 4 * 2;    // Output: 11 ->
```

```
let result = (3 + 4) * 2;  // Output: 14 -> Parentheses first
```

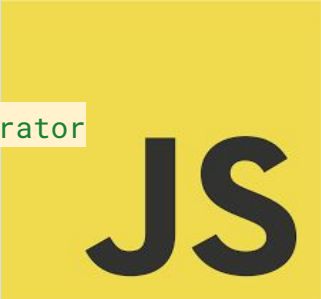
```
let result = 5 > 3 && 2 < 4; // Output: true -> first comparison then And operator
```

```
let a = 5; let b = 2;
```

```
let result = a + b * 3;
```

```
a = result; // Output: 11 -> first * then +
```

```
let result = true || false && false; // Output: true -> && then ||
```

A yellow square containing the letters 'JS' in a large, bold, black sans-serif font, representing JavaScript.



# Conditional Statements

Conditional statements execute code blocks based on conditions

→ `if` Statement:

- ◆ Executes `if` a condition is `true`.

→ `if - else` Statement:

- ◆ Executes `if` a condition is `true` or if it is `false`, `else` will execute

→ `if - else if - else` Statement:

- ◆ Executes `if` a condition is `true` or if it is `false`, then it checks for `else if`
- ◆ If `else if` is `true` or `else` will execute



# Conditional Statements

*if - else if - else Ladder:*

*In one if - else if - else family, we can have one if, one else, but we can have multiple else if (s), if needed,*



JS

*if*  
(only one)

*else - if (s)*  
(multiple as required)

*else*  
(only one)

# Conditional Statements

A yellow square with the letters 'JS' in a bold, black, sans-serif font, representing JavaScript.

The below are the few examples to understand the if - else if - else

if Example:

```
let age = 21;  
if (age >= 21) {  
  console.log("You can Marry.");  
}
```

if - else Example:

```
let age = 16;  
if (age >= 18) {  
  console.log("can vote.");  
} else {  
  console.log("cannot vote.");  
}
```

if - else if - else Example:

```
let marks = 75;  
if (marks >= 90) {  
  console.log("Grade A");  
}  
else if (marks >= 75) {  
  console.log("Grade B");  
}  
else {  
  console.log("Grade C");  
}
```

# Combination of All

In day-to-day programming or practical coding scenarios, all these concepts and operators are often combined and used together seamlessly.



## Let us Take A Simple Example As Follows

Write a program to determine if a customer is eligible for a discount. Declare variables such as **userName** for the customer's name, **totalOrderedValue** for the total value of the order, **minOrderValue** for the minimum order value required to qualify for the discount, and **discount** to represent the discount percentage.

The program should provide output in one of the following formats: **"Hurray, <userName>, a discount of <discount>% has been applied!"** if the customer *qualifies for the discount*, or **"Dear <userName>, please add items worth <balanceAmount> to your cart to qualify for a <discount>% discount."** if the customer *does not meet the minimum order value*.

# Combination of All.....

A simplest Solution for previous example:

Example:

```
//Variables
```

```
let userName = "Alice";
```

```
let totalOrderedValue = 800;
```

```
let minOrderValue = 1000;
```

```
let discount = 10;
```

```
// Check if the discount is applicable
```

```
if (totalOrderedValue >= minOrderValue) {
```

```
  console.log(`Hurray, ${userName}, a discount of ${discount}% has been applied!`);
```

```
  } else {
```

```
    let balanceAmount = minOrderValue - totalOrderedValue;
```

```
    console.log(`Dear ${userName}, please add items worth Rs.${balanceAmount} to your  
    cart to qualify for a ${discount}% discount.`);
```

```
  }
```

A yellow square with the letters 'JS' in a large, bold, black sans-serif font, representing JavaScript.